

Vasic-Digital-Reusable-Module-Suite

一度構築すれば、どこでも再利用——独立テスト済みの**Go**および**KMP**モジュール群による分散型システム

概要

digital.vasic.* (Go) およびKotlin Multiplatform名前空間の下で公開される、汎用的で再利用可能なモジュール群。各モジュールは独立しており、個別にテスト・バージョン管理され、より大規模なプロダクト（Catalogizer、HelixAgent、およびその他のシステム群）に対して同等のコードベースを持つサブモジュールとして利用される。このページでは、個別ページとしてはノイズとなる多数の小規模ユーティリティを一元化している。

短い説明

厳選されたdigital.vasic.*モジュール群——インフラ基盤（認証、キャッシュ、データベース、設定、可観測性）、AI/エージェント構築ブロック（RAG、VectorDB、埋め込み、MCP、エージェント機能、プランニング）、防御的LLMガードレール（RedTeam、Normalize）——に加え、Kotlin Multiplatformミラーセットも提供。いずれも汎用性が高く、テスト済みで再利用可能な設計となっている。

長い説明

vasic-digital組織は、一つの構造的な賭けに基づいて運営されている。「憲法+多数の独立した再利用可能モジュール」という哲学だ。汎用的な機能は二度と書かない。モノリシックな設計ではなく、再利用可能な機能はすべて小さなモジュールに切り出される——それぞれが独立したリポジトリ、テスト、ドキュメントを持ち、厳密に疎結合を保つことで、利用者固有の要件が一切入り込まない。このページでは、それらをまとめている。個別に見ればライブラリスケールの存在であり、単独のプロダクトページとしてはノイズに過ぎないからだ。しかし、全体として見れば、組

織の真の力を倍増させるものとなる。新しいプロダクトを「構築する」のではなく、「実績ある部品を組み立てる」という発想を可能にし、このモジュール群が「車輪の再発明をしない」という主張を具体的に裏付ける。

このスイートは三つのクラスターに分かれている。インフラ基盤 (Go) は、すべてのサービスに必要な土台を提供する: auth (JWT/bcrypt)、cache (Redis/TTL)、database (マイグレーション、デュアルSQLite/PostgreSQL)、config、middleware、observability (Prometheus/OpenTelemetry)、ratelimiter、security、storage (S3/MinIO)、streaming (WebSocketハブ)、eventbus、filesystem (マルチプロトコル)、discovery/mdns、http3、recovery、concurrency、lazyなど。**AI/エージェント構築ブロック** (Go) は、AIシステムの基盤を提供: rag、vectordb、embeddings、memory、conversation (無限コンテキスト圧縮、イベントソーシング)、mcp (Model Context Protocol)、toolschema、skillregistry、agentic (グラフベースのワークフロー編成)、planning (HiPlan/MCTS/Tree-of-Thoughts)、benchmark (SWE-bench/HumanEval/MMLU)、llmops、selfimprove (報酬モデリング/RLHF)、toon (Token-Oriented Object Notation)。防御的**LLM**ガードレールは、敵対的な環境に対する堅牢性を確保するツール群: RedTeam (YAML駆動の敵対的フィクスチャ)、Normalize (敵対的入力正規化)。さらに、**Kotlin Multiplatform** セットは、コアモジュール (Auth-KMP、Database-KMP、Security-KMP、UI-Components-KMPなど) をクロスプラットフォームアプリ向けにミラーリングしている。

コンテンツ

なぜこれを構築したのか

毎回ゼロから複数のプロダクト (Catalogizer、HelixAgent、Heraldなど) を作り上げるのは無駄が多く、一貫性にも欠ける。汎用的な機能をすべて切り離し、テスト済みの独立モジュールとして抽出することで、修正や改善が全体に波及し、新しいプロダクトも実績ある部品の組み合わせで構築できる。

なぜゲームチェンジャーなのか

これは事実上、AIを中心としたバックエンド構築のための「プライベート標準ライブラリ」であり、多くのチームが手をつけられない層だ。なぜなら、認証、キャッシュ、RAGの基盤作りを五度も繰り返すことに追われているから。ここではインフラの基本要素、AIの構成要素、防御的なLLMのガードレールがすべて、独立してテストされたドロップインモジュールとして存在する。これにより、小規模なチームでも通常なら大規模なチームが必要なペースでプロダクトレベルのシステムを構築でき、その過程で通常発生する重複の負債も蓄積しない。

革新的な点

- フリート全体の分離原則（CONST-051）：サブモジュールは独立したコードベースとして扱い、利用者固有の要素を一切含まない。
- AIの基本レイヤー（RAG、VectorDB、エンベディング、MCP、ToolSchema、エージェント、プランニング、LLMOps）を再利用可能なモジュールとして提供。
- 防御的LLMガードレールクラスター（RedTeam、Normalize）による敵対的ロバスト性の確保。
- 同一の規約に基づく並列Go + Kotlin Multiplatformモジュールセット。

課題と解決策

- 数十のモジュールにわたる結合の劣化：「憲法」による分離契約と、実行時の利用者固有要素の注入で解決。
- 多数のモジュールの一貫性とテストの維持：モジュールごとのテスト・ドキュメント・課題に共通の規約を適用し、HelixConstitutionのガバナンス基盤で解決。
- クロスプラットフォーム対応：コアモジュールのKotlin Multiplatformミラー版で解決。

テックスタック（理由と方法）

- **Go** — ほとんどのモジュール（digital.vasic.*）。
- **Kotlin Multiplatform** — クロスプラットフォーム対応のミラーモジュール（認証/データベース/セキュリティ/UI/並行処理/レートリミッターKMP）。
- **Redis / PostgreSQL / SQLite** — キャッシュ、データベース、ストレージの基本要素。
- **Prometheus / OpenTelemetry** — 可観測性モジュール。
- **WebSocket / HTTP/3 (quic-go) / mDNS** — ネットワーキングモジュール。
- **Vector DB / embeddings / RAG / MCP** — AIの基本モジュール。
- **YAML** — RedTeamの敵対的フィクスチャと設定。

未検証 / 作業中：いくつかの組織リポジトリは「SCAFFOLD / WIP」と自己申告（例：PliniusCommon、I-LLM、HyperTune、AutoTemp、Veritas、Ouroborous、Claritas、LeakHub、GandalfSolutions）。これらは初期段階/スキャフォールドとして扱い、リリース済みではない。